



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE COMPUTO
INGENIERÍA EN SISTEMAS



3CV4 – Paradigmas de programación

ESTUDIANTES:

BONILLA HERNANDEZ XIMENA SOFIA

2026630037

CARRASCO VILLEGAS RICARDO SEBASTIAN

2026630034

CRUZ RODRÍGUEZ BRUNO AARÓN

2026630033

PROFESORA: GUADALUPE ANA GABRIELA MARTINEZ HERNANDEZ

02 – OCTUBRE – 2026

1. Recuperación del Trabajo Previo

Descripción del problema:

Necesitamos un gestor de citas para un hospital que ayude a los doctores, pacientes y personal administrativo a tener un mejor manejo y control de las citas medicas de las diferentes especialidades que tiene el hospital las cuales son: Medicina general, pediatría, ginecología, Obstetricia, Neurología, cardiología y Ortopedia. Este gestor de citas ayudara a los pacientes a agendar citas de una manera mas sencilla, a los doctores y recepcionistas a revisar y monitorear las citas con más precisión.

Requerimientos

Paciente:

El usuario no puede generar citas si el horario de todos los doctores está ocupado
No puede agendar más de una cita en la misma área
Solo se puede cancelar la cita un día antes de la consulta
No se puede agendar citas en horario no laboral
Debe estar logeado para generar una cita
No se puede generar dos o más citas en un mismo horario sin importar que sea en diferentes áreas
El usuario puede ver los doctores disponibles, horarios disponibles y áreas donde se encuentran sin iniciar sesión
El usuario solo puede cambiar el horario de las citas 24 horas antes y si está disponible el horario en otro día
Para poder registrarse por primera vez se necesitan los siguientes datos: Nombre, CURP, Correo, Contraseña, Genero.
El sistema notificara al paciente si el doctor tuvo un percance y esta deshabilitado en sus citas

Doctor:

Solo el doctor y el administrador pueden ver la bitácora del paciente.
El doctor puede ver su horario de citas

Recepcionista:

El recepcionista solo puede ver los horarios de cita de los doctores
El recepcionista puede deshabilitar el horario de un doctor si este no puede realizar las citas del día

Administrador:

Solo el administrador puede modificar los horarios de los doctores
Solo el doctor y el administrador pueden ver la bitácora del paciente
Solo el administrador puede dar de alta los perfiles del personal en el sistema para su acceso

Roles y actores.

- **Administrador:** Va a administrar y controlar el sistema.
- **Usuario:** Va a solicitar y gestionar sus citas médicas.
- **Doctor:** Va a gestionar sus citas y atender a sus pacientes.
- **Recepcionista:** Va a gestionar las citas y registrar a los pacientes

Casos de uso.

- Agendar una cita
- Cancelar una cita
- Mover una cita
- Iniciar sesión
- Cerrar sesion
- Ver cita
- Consultar cita
- Ver doctores
- Consultar especialidades

- Vista de bitácora

Elección del lenguaje.

Python.

Enlazar la base de datos en SQL server con el frontend de JavaScript, además de la programación orientada a objetos.

Criterios de evaluación.

- Sintaxis, es la mas sencilla que hay.
- Semántica, es la mas facil de interpretar.
- Tipado, no mezcla datos de diferentes tipos, dinamico.
- Manejo de memoria, tiene una excelente gestion de memoria, asigna y libera sin programarlo.
- Estructuras de control, tiene las estructuras necesarias para el proyecto.
- Abstracciones y entorno, tiene funciones mas simples.

SQL

Base de datos

- Sintaxis: La sintaxis es bastante simple ya que usa palabras en inglés muy intuitivas tanto para crear tablas como para hacer consultas.
- Semántica Comprueba si los nombres de las tablas, tipos de datos, etc son correctos
- -ipado: cada columna tiene que tener un tipo de dato
- Manejo de memoria: Gestión de forma dinámica
- Estructuras de control No tiene
- Abstracciones y entorno Simplifica el acceso a los datos, restricción de privilegios

CSS

- Sintaxis: solo se compone de bloques encerrados con llaves
- Semántica: Dar diseño a HTML
- Tipado: No existe
- Manejo de memoria: No tiene
- Estructuras de control: Bloques de declaración
- Abstraccion y entorno: separar los estilos de cada componente

HTML

- Sintaxis: uso de etiquetas
- Semántica: describen claramente el significado del contenido que contienen
- Tipado: No tiene
- Manejo de memoria: no maneja
- Estructura de control: no tiene
- Abstraccion y entorno: es intuitivo

JavaScript

- Sintaxis, flexibilidad y similitud con otros lenguajes populares.
- Semántica, se enfoca en mejorar, manipular y extender, el significado de html
- Tipado, permite mezclar tipo de datos de forma implícita
- Manejo de Memoria, tiene una excelente gestion de memoria, asigna y libera sin programarlo
- Estructuras de control, tiene las estructuras de control necesarias para el proyecto
- Abstracciones y entorno, Tiene funciones mas faciles

2. Selección de una Funcionalidad

Requerimiento: Solo se puede cancelar la cita un día antes de la consulta

1.- ¿Qué problema resuelve y quién la utiliza?

Ayuda a verificar que la cita sea cancelada con la anticipación correcta para que se puedan hacer modificaciones, recorrer citas, etc... y así evitar que los horarios cancelados queden inutilizables para otras personas.

Esto lo usara el paciente, pero beneficiara el tiempo del doctor

2.- ¿Con qué requerimientos y casos de uso se relaciona?

Se relaciona con agendar citas, cancelar citas, mover cita, vista de bitácora y con que solo se puede cancelar la cita un día antes de la consulta

3.-¿Cuáles son sus entradas, transformaciones y salidas?

- **Entrada:** Nombre, Fecha, Estado
- **Salidas:** Estado de la cita

4.- ¿Por qué es adecuada para el paradigma funcional y qué ventajas o limitaciones presenta frente a una solución imperativa?

Se adecua mas porque es solo una regla matemática pura recibe la fecha actual y la fecha de la cita, calcula si hay al menos 24 horas de diferencia o un día y regresa un valor Booleano sin modificar nada fuera de la función

- Ventajas:
- Facilidad para hacer pruebas
- Inmutabilidad
- Verificabilidad
- Limitaciones:
- Mas uso de memoria
- Un poco mas complejo

Pseudocodigo

```
// Función lambda pura para verificar regla de negocio (>= 24 hrs)
Definir es_cancelable como lambda(cita, fecha_actual):
    Retornar (cita.fecha - fecha_actual) >= 1_dia

// Función lambda pura para inmutabilidad de datos
Definir actualizar_estado_cita como lambda(cita, nuevo_estado):
    Retornar clonar_registro_con_cambio(cita, "estado": nuevo_estado)

// Función de orden superior (Uso de 'map' y cálculo lambda)
Definir procesar_cancelacion como función(citas, id_cita, fecha_actual):
    Retornar map(
        lambda cita:
            Si (cita.id == id_cita Y es_cancelable(cita, fecha_actual)) Entonces
                actualizar_estado_cita(cita, "Cancelada")
            Sino
                cita
            FinSi,
        citas
    )
```

Inicio

```
fecha_actual = Obtener_Fecha_Del_Sistema()
fecha_maxima = fecha_actual + 5_años

Imprimir "--- SISTEMA DE GESTIÓN DE CITAS (FUNCIONAL) ---"
Imprimir "Fecha actual del sistema: ", fecha_actual

mis_citas = []

// Captura manual de citas (ej. 3 iteraciones)
Para i desde 1 hasta 3 Hacer
    Imprimir "--- Cita ", i, " ---"
    paciente = Leer("Nombre del paciente: ")

    // Bucle de validación de fecha
    Mientras Verdadero Hacer
        Intenta
            fecha_str = Leer("Fecha de la cita (AAAA-MM-DD): ")
            fecha_cita = Convertir_Texto_A_Fecha(fecha_str)

            // Caso P03: Citas no pueden ser mayores a 5 años
            Si fecha_cita > fecha_maxima Entonces
                Imprimir "Error: Vuelve a pedir la cita (excede los 5 años).\"
                Continuar_Iteracion_Actual
            FinSi

            // Casos P04 y P08: Formato y día bisiestro válidos
            Imprimir "La cita se guarda exitosamente. Lo guarda normalmente como: ", fecha_str
```

```
Romper_Bucle // La fecha es correcta, sale del Mientras

Capturar Error_De_Formato
    // Caso P07: Fechas inexistentes (ej. 29 de febrero no bisiesto)
    Imprimir "Error: Ese día no existe o el formato es incorrecto."
FinIntenta
FinMientras

estado = Leer("Estado de la cita (Activa / Cancelada): ")
Si estado NO ES "Activa" Y estado NO ES "Cancelada" Entonces
    estado = "Activa"
FinSi

cita = {id: i, paciente: paciente, fecha: fecha_cita, estado: estado}
Añadir cita a la lista mis_citas
FinPara

// Mostrar citas registradas
Imprimir "--- LISTA DE CITAS REGISTRADAS ---"
Para cada c en mis_citas Hacer
    Imprimir c.id, c.paciente, c.fecha, c.estado
FinPara

estado_actual_citas = mis_citas // Evitar mutar el arreglo original
continuar = "s"

Mientras continuar == "s" Hacer
    id_a_cancelar = Leer_Entero("Ingresa el ID de la cita que deseas cancelar: ")

    // Buscar la cita usando funciones de búsqueda en colecciones
    cita_objetivo = Buscar_Cita_Por_ID(estado_actual_citas, id_a_cancelar)

    Si cita_objetivo ES Nulo Entonces
        // Caso P09: Búsqueda de ID inexistente
        Imprimir "El programa detecta que ese id no existe y pide otro."
        Continuar_Iteracion_Actual

    Sino Si cita_objetivo.estado == "Cancelada" Entonces
        // Caso P01: Cita ya cancelada
        Imprimir "No se puede cancelar."

    Sino Si cita_objetivo.fecha < fecha_actual Entonces
        // Caso P10: Cita en el pasado
        Imprimir "Esta cita ya fue atendida."

    Sino
        // Procesamiento funcional puro de actualización de estado
        estado_actual_citas = procesar_cancelacion(estado_actual_citas, id_a_cancelar,
fecha_actual)
```

```
// Comprobación posterior de éxito
cita_nueva = Buscar_Cita_Por_ID(estado_actual_citas, id_a_cancelar)

Si cita_nueva.estado == "Cancelada" Entonces
    // Casos P02 y P06: Cambio exitoso (>= 24 hrs)
    Imprimir "Se cancela correctamente. / Lo cancela normalmente."
Sino
    // Caso P05: Rechazo funcional por tiempo (< 24 hrs)
    Imprimir "No lo cancela."
FinSi
FinSi

continuar = Leer("¿Deseas intentar cancelar otra cita? (s/n): ")
FinMientras

// Mostrar estado final
Imprimir "--- ESTADO FINAL DE LAS CITAS ---"
Para cada c en estado_actual_citas Hacer
    Imprimir c.id, c.paciente, c.fecha, c.estado
FinPara
Fin
```

```
Codigo Python
from datetime import datetime, timedelta

es_cancelable = lambda cita, fecha_actual: (cita["fecha"] - fecha_actual) >= timedelta(days=1)

actualizar_estado_cita = lambda cita, nuevo_estado: {**cita, "estado": nuevo_estado}

def procesar_cancelacion(citas: list, id_cita: int, fecha_actual: datetime) -> list:
    return list(
        map(
            lambda cita: actualizar_estado_cita(cita, "Cancelada")
            if cita["id"] == id_cita and es_cancelable(cita, fecha_actual)
            else cita,
            citas
        )
    )

fecha_actual = datetime.now()
fecha_maxima = fecha_actual + timedelta(days=365 * 5)

print(f"--- SISTEMA DE GESTIÓN DE CITAS (FUNCIONAL) ---")
print(f"Fecha actual del sistema: {fecha_actual.strftime('%Y-%m-%d')}\n")

mis_citas = []

print("Por favor, ingresa los datos para las citas:")
for i in range(1,2):
    print(f"\n--- Cita {i} ---")
    paciente = input("Nombre del paciente: ")

    while True:
        try:
            fecha_str = input("Fecha de la cita (formato AAAA-MM-DD): ")
            fecha_cita = datetime.strptime(fecha_str, "%Y-%m-%d")

            if fecha_cita > fecha_maxima:
                print("Error: Vuelve a pedir la cita (excede los 5 años).")
                continue

            print(f"La cita se guarda exitosamente. Lo guarda normalmente como: {fecha_str}")
            break
        except ValueError:
            print("Error: Ese día no existe o el formato es incorrecto.")

    estado = input("Estado de la cita (Activa / Cancelada): ").capitalize()
    if estado not in ["Activa", "Cancelada"]:
        estado = "Activa"

    mis_citas.append({
```

```
"id": i,
"paciente": paciente,
"fecha": fecha_cita,
"estado": estado
})

print("LISTA DE CITAS REGISTRADAS:")
for c in mis_citas:
    print(f'ID: {c['id']:2} | Paciente: {c['paciente']:15} | Fecha: {c['fecha'].strftime('%Y-%m-%d')} |
Estado: {c['estado']}')
print("PROCESO DE CANCELACIÓN")

estado_actual_citas = mis_citas
continuar = "s"

while continuar.lower() == "s":
    try:
        id_a_cancelar = int(input("\nIngresa el ID de la cita que deseas cancelar: "))

        cita_objetivo = next((c for c in estado_actual_citas if c["id"] == id_a_cancelar), None)

        if cita_objetivo is None:
            print("El programa detecta que ese id no existe y pide otro.")
            continue

        elif cita_objetivo["estado"] == "Cancelada":
            print("No se puede cancelar.")

        elif cita_objetivo["fecha"] < fecha_actual:
            print("Esta cita ya fue atendida.")

        else:
            estado_actual_citas = procesar_cancelacion(estado_actual_citas, id_a_cancelar,
fecha_actual)
            cita_nueva = next((c for c in estado_actual_citas if c["id"] == id_a_cancelar), None)

            if cita_nueva["estado"] == "Cancelada":
                print("Se cancela correctamente. / Lo cancela normalmente.")
            else:
                print("No lo cancela.")

    except ValueError:
        print("Por favor, ingresa un número válido.")

    continuar = input("\n¿Deseas intentar cancelar otra cita? (s/n): ")

print("ESTADO FINAL DE LAS CITAS:")
for c in estado_actual_citas:
    print(f'ID: {c['id']:2} | Paciente: {c['paciente']:15} | Fecha: {c['fecha'].strftime('%Y-%m-%d')} |
Estado: {c['estado']}')
```

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
	Si el estado de la cita es "cancelado"no	Estado:			
P01	se puede cancelar	cancelado	No se puede cancelar	no se puede cancelar	aprobada
	Cancelación con exactamente 24h de	Fecha:			
P02	anticipación	2026-10-03	Se cancela correctamente	no lo cancela	fallida
		Fecha:		error:excede los 5 años	
P03	Las citas no pueden ser mayor a 5 años	2040-04-02	Vuelve a pedir la cita	y vuelve a pedir la cita	aprobada
	El formato de la fecha de la cita tiene	Fecha:		guarda la fecha	
P04	que estar AAAA-MM-DD	2026-12-20	Lo guarda normalmente como: 2026-12-20	correctamente	aprobada
	La cita no se puede cancelar con	Fecha:			
P05	menos de 24h de anticipación	2026-10-02	No lo cancela	no lo cancela	aprobada
	La cita se puede cancelar con 24h o	Fecha:		la cita se cancela	
P06	mas de anticipación	2026-10-07	Lo cancela normalmente	exitosamente	aprobada
	No se puede agendar una cita el 29 de	Fecha:			
P07	febrero de un año no bisiestos	2027-02-29	Ese día no existe	no acepta la fecha	aprobada
	se puede agendar una cita el 29 de	Fecha:		guarda la fecha	
P08	febrero de un año bisiesto	2028-02-29	La cita se guarda exitosamente	correctamente	aprobada
P09	Busqueda de id inexistente	ID: 99	El programa detecta que ese id no existe y pide otro	no existe y pide otro	aprobada
	Si la fecha de la cita ya paso no se	Fecha:		la cita ya paso no puede	
P10	puede cancelar	2026-06-15	Esta cita ya fue atendida	ser cancelada	aprobada